

URL Shortener — Functional Requirements

1. Shorten a Link

Given a long URL, return a compact alias the user can share instead.

2. Resolve a Link

Visiting the short link sends the browser to the original destination via a redirect.

3. Vanity Slugs

Logged-in users may request a memorable custom slug instead of a random one, subject to availability.

4. Auto-Expiry

A link can optionally be created with a TTL; after that window it stops resolving.

5. Traffic Insights

Owners can see visit counts, time-of-visit trends, referring sites, and rough visitor geography per link.

6. Link Dashboard

Signed-in users can list, rename, update, pause or permanently remove links they own.

7. Idempotent Submits

Re-submitting a long URL that a user already shortened returns their existing short link instead of minting a new one.

8. Abuse Throttling

Cap how many links a single account or IP can mint within a rolling time window.

9. QR Code Export

Any short link can be rendered as a downloadable QR code for offline/print use.

10. Malicious-Link Screening

Submitted destinations are checked against a threat-intel blocklist before a short link is issued.

URL Shortener — Non-Functional Requirements

1. Near-Continuous Uptime

Target four nines for the redirect path; the creation path can tolerate brief blips without user-visible impact.

2. Snappy Redirects

p99 redirect latency should stay well under 100ms — this call sits directly in the user's click path.

3. Elastic Scale-Out

Every tier scales horizontally to absorb billions of stored links and traffic spikes in the tens of thousands of req/sec.

4. No Silent Data Loss

Once a short link is acknowledged as created, it must survive node failures, disk failures, and AZ outages.

5. Read-Dominant Workload

Clicks vastly outnumber creations (assume roughly 100 reads per write) — design for cheap, cache-friendly reads.

6. Non-Guessable Codes

Short codes must not follow a predictable sequence, or attackers could enumerate private links.

7. Defense in Depth

Screen outbound destinations for known-bad domains and protect accounts with salted/hashed credentials and token auth.

8. Mixed Consistency Model

Short-code uniqueness needs strong consistency at write time; click counters and dashboards can be eventually consistent.

URL Shortener — Back-of-the-Envelope Estimation

Assumptions

- 200M new short links created per month
- Read:Write ratio $\approx 100:1 \rightarrow \sim 20B$ redirects/month
- Retain link + click data for 3 years
- Average stored record ≈ 300 bytes (link row); click event ≈ 200 bytes

Write QPS

$200,000,000 / (30 \times 24 \times 3600)$
 ≈ 77 links/sec (average)
Peak ($\times 4$) ≈ 310 links/sec

Read (Redirect) QPS

$77 \times 100 \approx 7,700$ req/sec (average)
Peak ($\times 4$) $\approx 30,800$ req/sec

Link Storage (3 years)

$200M/mo \times 36 \text{ months} \times 300B$
 $\approx 7.2B$ links $\times 300B$
 ≈ 2.2 TB total

Incoming Bandwidth (writes)

$77 \times 300B \approx 23$ KB/sec

Outgoing Bandwidth (redirects)

$7,700 \times 300B \approx 2.3$ MB/sec

Cache Memory (hot set)

Assume 15% of daily reads are hot
 $7,700 \times 86400 \times 0.15 \times 300B$
 ≈ 30 GB $\rightarrow$ round up to ~ 40 GB

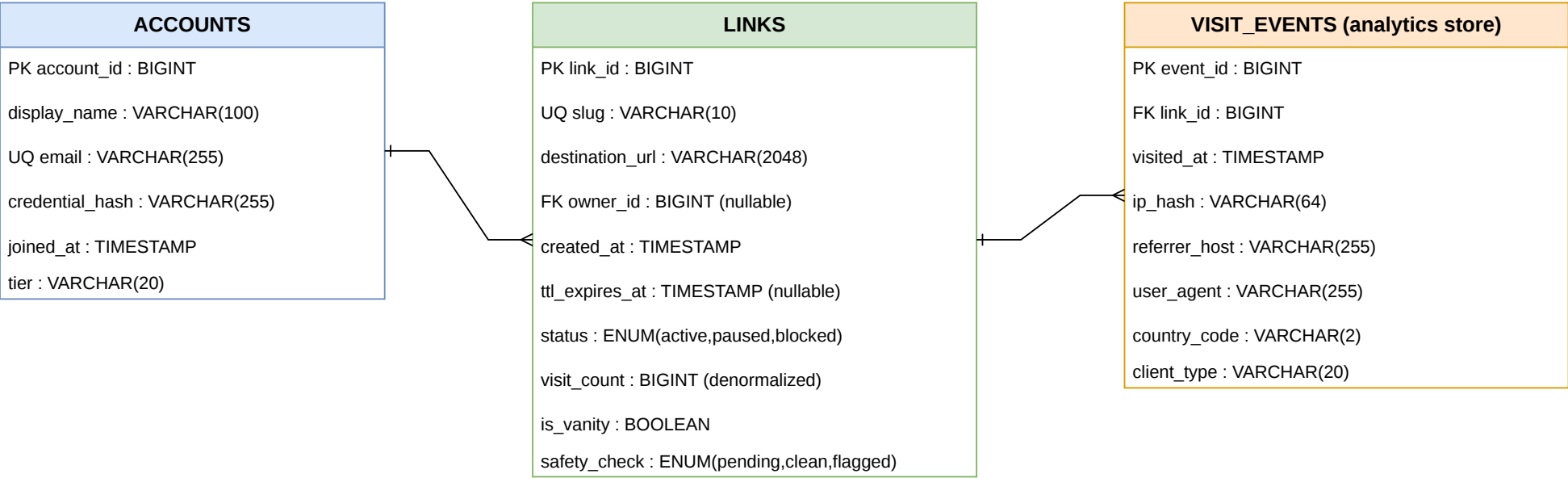
Key-space Sizing

Base62 alphabet [a-zA-Z0-9], 62 symbols
6 chars $\rightarrow 62^6 \approx 56.8$ billion unique codes
Comfortably covers 7.2B links with room to grow

Fleet Sizing

Assume 1 redirect instance handles $\sim 1,200$ req/sec
Peak 30,800 req/sec $\rightarrow \sim 26$ instances
(rounded up, spread across 3 AZs for headroom)

URL Shortener — Database Design (ER Diagram)



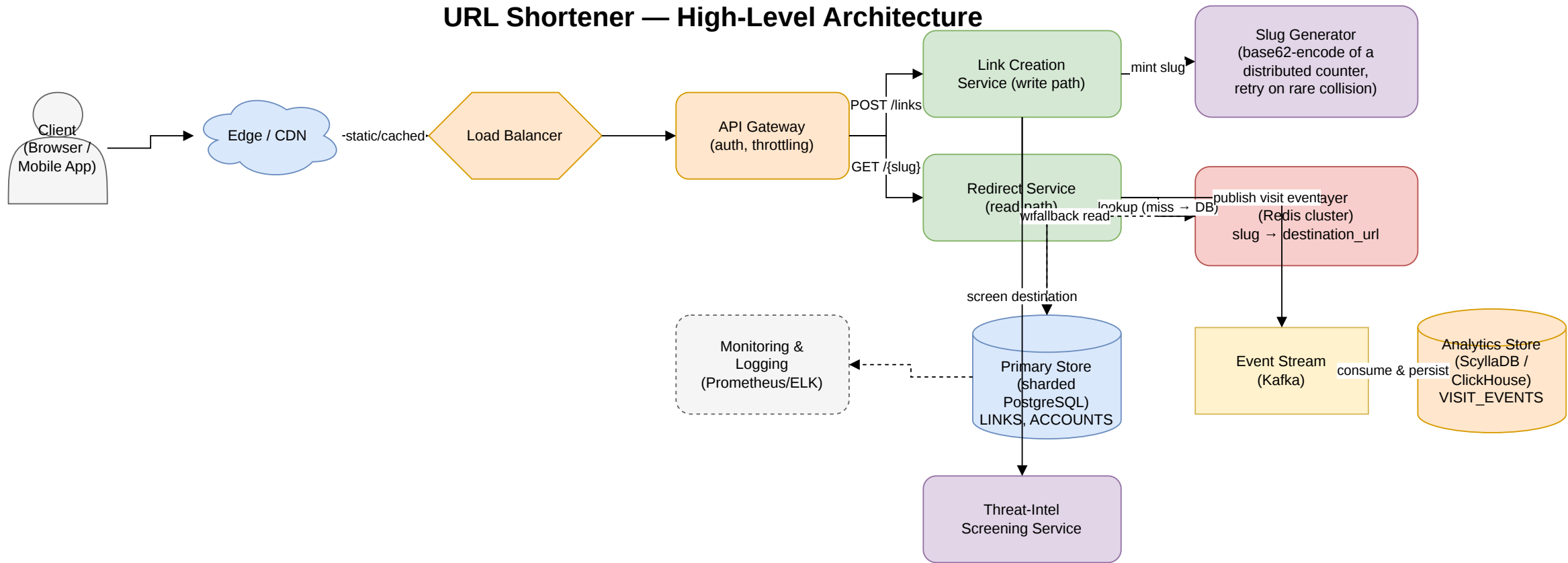
Notes:

- LINKS table: relational store (e.g. PostgreSQL), range- or hash-sharded on slug so lookups route to one shard.
- VISIT_EVENTS: append-only, very high write volume — favors a wide-column / time-series store (e.g. ScyllaDB) or a stream → OLAP pipeline (e.g. Kafka → Druid/ClickHouse).
- visit_count on LINKS is a rolling counter updated asynchronously so the redirect path never blocks on analytics writes.
- owner_id is nullable to support anonymous, unauthenticated link creation.

URL Shortener — API Design

POST	<code>/api/v1/links</code> Create a short link. Body: { destination_url, vanity_slug?, ttl_seconds? } → 201 { slug, short_url }
GET	<code>/slug</code> Resolve and redirect to the destination URL. 302 Found with Location header; 410 Gone if expired/paused, 404 if unknown.
GET	<code>/api/v1/links/{slug}</code> Fetch link metadata (destination_url, created_at, ttl_expires_at, visit_count, status). Owner-only.
PATCH	<code>/api/v1/links/{slug}</code> Partially update a link: change destination, TTL, or status (active/paused). Owner-only.
DELETE	<code>/api/v1/links/{slug}</code> Permanently remove a link (owner-only) → 204 No Content.
GET	<code>/api/v1/links/{slug}/stats</code> Traffic insights: total visits, visits-over-time buckets, top referrers, top countries.
GET	<code>/api/v1/accounts/{accountId}/links</code> List links owned by an account, paginated (?cursor=, ?limit=).
GET	<code>/api/v1/links/{slug}/qrcode</code> Return a PNG/SVG QR code encoding the short URL.
POST	<code>/api/v1/auth/signup</code> Create a new account. Body: { display_name, email, password }.
POST	<code>/api/v1/auth/session</code> Authenticate and issue a short-lived JWT plus refresh token.
Notes: All endpoints except redirect, signup, and session require Authorization: Bearer . Errors follow { error_code, message, request_id }. Throttling is enforced at the API Gateway, keyed by account and by IP.	

URL Shortener — High-Level Architecture



Write path: Client → Edge/CDN → LB → API Gateway → Creation Service → Slug Generator (base62 counter, collision retry) → Threat-Intel Screening → Primary Store.

Read path: Client → Edge/CDN → LB → API Gateway → Redirect Service → Cache (hit) or Primary Store (miss, then warm the cache) → 302 redirect.

Visit events are published asynchronously to the event stream and persisted to a separate analytics store, keeping the redirect path free of analytics writes.